

Microblaze Software Sorter

Goal

Implement **software** sorting routine on MicroBlaze that moves data through the provided AXI-Stream DMA path. You should optimize algorithm so that latency is consistently below 400us. Data size and width is known. Each chunk is 256 32-bit words representing unsigned integer.

System Architecture

Architecture consists of:

- Microblaze Microcontroller- processing unit
- Other IP blocks- can be deduced from Vivado project.

Detailed Description

This is purely software task, please note that:

- Competitors are not allowed to modify any hardware provided in task_8 repository nor delete hardware components from .tcl file!
- Competitors are only allowed to modify software components from task_8/sw directory.

Input data to be sorted is driven to input in ascending order. Competitors' task is to receive input data, sort it, and push it to the output.

Input format:

- 256 32-bit words representing unsigned integer

Output format:

- 256 32-bit words representing unsigned integer

Users are encouraged to use reference:

- Microblaze: [Introduction • MicroBlaze Processor Reference Guide \(UG984\) • AMD Technical Information Portal](#)
- AXI DMA: [Introduction • AXI DMA LogiCORE IP Product Guide \(PG021\) • AMD Technical Information Portal](#)

Points

Competitors are assigned:

- 0 points for not sorted array or lack of output
- 10 points for properly sorted array and task latency > 400us
- 20 points for properly sorted array and task latency < 400us

Building Software

- **Enter Software directory:** `cd src/task/task_8/sw`
- **Run bootstrap script:** `vitis -s bootstrap.py`
- **Build Software:** `vitis -s build.py`

Please note that the last step must be executed after each change. Competitors don't need to push .elf file. Changing main.c and possibly CMakeLists.txt is sufficient.

Running Testbench

Testbench is auto associated with .elf file present in build location. When running simulation for task_8 user will also be simulating software. There is no possibility of debugging software in a classical way. All the information must be gathered from waveform analysis. Please note that simulation must be re-run after each rebuilt.